Qualcomm AI research

MOCCASIN: Efficient Tensor Rematerialization for Neural Networks

Burak Bartan[1] / Haoming Li[2] / Harris Teague[1] / Christopher Lott[1] / Bistra Dilkina[2]

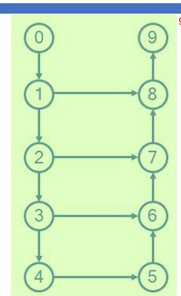
[1]Qualcomm AI Research, [2]University of Southern California

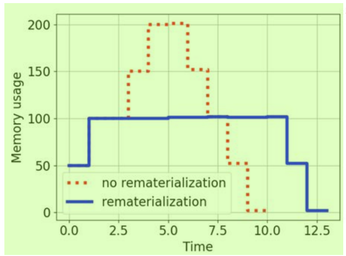
bbartan@qti.qualcomm.com

1. Introduction

- The deployment and training of neural networks on edge computing devices pose many challenges.
- The low memory nature of edge devices is often one of the biggest limiting factors encountered in the deployment of large neural network models.
- Tensor rematerialization or recompute is a way to address high memory requirements for neural network training and inference.
- In this paper we consider the problem of execution time minimization of compute graphs subject to a memory budget.
- In particular, we develop a new constraint programming formulation called Moccasin with only $O(n)$ integer variables, where n is the number of nodes in the compute graph (as opposed to prior work formulations with $O(n^2)$ variables).

2. Motivation and problem statement





MEMORY-CONSTRAINED COMPUTATION GRAPH SEQUENCING WITH REMATERIALIZATION

$$\underset{seq(G)}{\text{minimize}} \quad \text{total execution duration}$$

subject to

- $seq(G)$ meets the data dependencies of G
- peak memory footprint of $seq(G)$ $\leq$ local memory capacity M

Consider the example 10-node computation graph above, and assume:

1. Output sizes are 50 for nodes on the left, and 1 for the ones on the right.
2. Duration is 1 for all nodes.

Consider execution of the nodes in the order

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

We need to keep the output of node 1 in memory until we execute node 8 (similarly for 2&7)

Rematerialization: Discard output of node 1 after node 2 is computed and recompute it later:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

discard 1 recompute 1

To recompute node 1: We need to recompute 0 or have it available in memory – another decision that needs to be made by the algorithm

3. Prior work

The work of [1] develops a mixed integer linear programming (MILP) formulation to solve this problem.

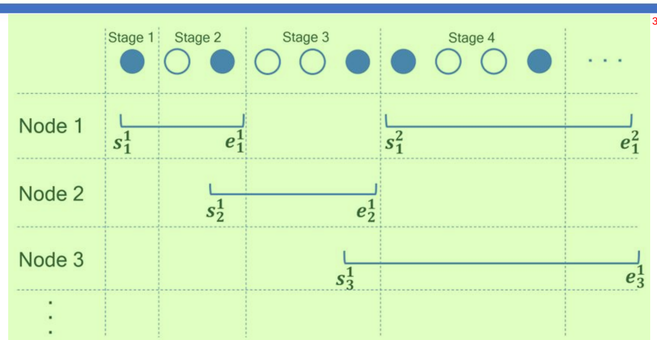
- Introduces Boolean matrices R, S, F for recompute, storage, memory deallocation → $O(n^2 + nm)$ Boolean decision variables (n: # nodes, m: # edges)
- Poor scalability to large computation graphs

We have developed a new optimization problem formulation keeping scalability in mind, which greatly simplifies the problem.

Key assumption: Don't rematerialize any node more than C times.

[1] Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization, Proceedings of Machine Learning and Systems 2 (MLSys 2020). Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, Ion Stoica

4. Solution



Our solution is an optimization problem formulation where

- The main building block is the concept of output retention intervals which simplifies the problem formulation greatly
- Output retention intervals are used to model how long the output of a node is retained in memory
- We define C retention intervals for each node where each interval represents a rematerialization (i.e. recompute) of that node
- The starting s_v^i and end e_v^i times of the intervals are modeled as the decision variables of the optimization problem
- We then define the precedence and memory constraints using these decision variables
- The resulting problem could be posed as a constraint programming (CP) problem, which we could use any generic CP solver to solve numerically

5. Formal specification

$$\underset{s,e,a}{\text{minimize}} \quad \sum_{v,i} w_v a_v^i \qquad (1)$$

subject to

$$s_v^i \leq e_v^i, \forall v, i \qquad (2)$$

$$e_v^i \leq s_v^{i+1}, \forall v, \forall i < C_v \qquad (3)$$

$$\sum_{v,i\,:\,s_v^i \leq t \leq e_v^i} m_v a_v^i \leq M, \forall t \in \mathcal{D} \qquad (4)$$

$$\forall (u,v) \in E, \forall i \in \{i : a_v^i = 1\}, \exists j \text{ such that}$$

$$a_u^j = 1, s_u^j + 1 \leq s_v^i \leq e_u^j \qquad (5)$$

$$s_v^i \neq s_u^j, \forall v, u \in \{v, u : v \neq u\}, \forall i, j \qquad (6)$$

$$a_v^1 = 1, \forall v \qquad (7)$$

$$s_v^i, e_v^i \in \mathcal{D}, a_v^i \in \{0,1\}, \forall v, i. \qquad (8)$$

- (1) → Total amount of computation
- (2) → Start of an interval comes before end
- (3) → Ordering among the intervals of a node
- (4) → Memory use cannot exceed M
- (5) → Precedence constraints
- (6) → Interval start times are all different
- (7) → There is at least one active interval per node
- (8) → Variable domains

6. Numerical results

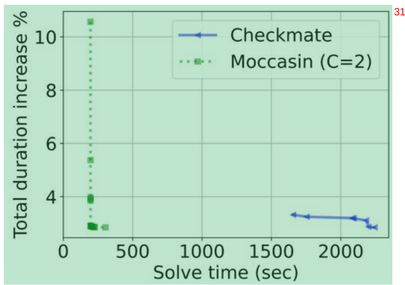


Figure: Percentage of total duration increase against solve time in seconds for our proposed method (Moccasin) and Checkmate [1] on a real-world graph with n=442 nodes and m=1247 edges.

Graph	(n, m)	M	Checkmate MILP TDI %	Checkmate MILP Peak mem	Checkmate MILP Time (s)	Checkmate LP+Rounding TDI %	Checkmate LP+Rounding Peak mem	Checkmate LP+Rounding Time (s)	Moccasin TDI %	Moccasin Peak mem	Moccasin Time (s)
RL G_2	(250, 944)	132,156	0.9	132,130	685.1	93.0	178,200	401.7	0.9	131,831	**55.0**
		117,472	-	-	-	328.6	181,200	696.9	4.9	117,400	**639.5**
RL G_4	(1000, 5875)	547,757	-	-	-	-	-	-	0.7	547,660	**3612.9**
		486,895	-	-	-	-	-	-	3.4	486,880	**3611.8**
RW 1	(358, 947)	20,227,276	2.3	20,226,048	1340.0	-	-	-	2.3	20,226,048	**123.5**
		17,979,801	4.5	17,977,344	1605.4	-	-	-	4.5	17,977,344	**122.0**
RW 2	(442, 1247)	10,817,740	1.4	10,811,392	1856.7	-	-	-	1.4	10,813,440	**1201.3**
		9,615,769	2.8	9,615,360	2242.9	-	-	-	2.8	9,613,312	**303.9**
RW 3	(574, 1304)	10,539,417	-	-	-	-	-	-	0.8	10,539,008	**1802.4**
		9,368,371	-	-	-	-	-	-	1.6	9,367,552	**1802.8**
CM 1	(73, 149)	11.3 GB	0.0	11.1 GB	6.3	0.0	11.4 GB	6.8	0.0	11.1 GB	**3.1**
		10.0 GB	0.1	9.65 GB	5.6	0.1	10.8 GB	6.7	0.1	9.9 GB	**3.1**
CM 2	(353, 751)	31.9 GB	**0.1**	31.6 GB	434.1	0.1	31.5 GB	505.2	0.2	31.9 GB	**65.2**
		28.4 GB	0.3	28.3 GB	485.3	0.5	27.8 GB	1065.4	0.3	28.4 GB	**69.3**

Table: Performance comparison across different graph families under different memory budgets.